Critical Analysis and Comparison of State Machines

Martin Krüger
martin.krueger@uni-ulm.de
Universität Ulm
Ulm, Germany

Abstract

State Space Models have emerged as a novel paradigm for sequence-to-sequence learning, positioning themselves as competitors to established RNNs and Transformers. This work provides a comprehensive introduction to SSM architectures, specifically S4 and Mamba. First, the architectural foundations of SSMs are explored and compared. Their theoretical strengths lie in dual computation modes: convolutional representation for parallel training and recurrent mode for efficient inference. Second, S4 and Mamba are critically analyzed based on published empirical benchmarks including Long Range Arena, language modeling, and machine translation. Findings show that S4 and Mamba generally perform strongly but do not consistently outperform state-of-the-art models. Performance proves highly task-specific and heavily influenced by benchmark methodology and application domain. SSMs tend to excel on extreme-length sequences due to inference efficiency but struggle with tasks demanding fine-grained content-based reasoning, like machine translation. This work concludes that SSMs complement Transformers in sequence modeling, rather than replacing them.

1 Introduction

Sequence-to-Sequence learning gained significant traction in 2014 with two influential papers by Sutskever et al. and Cho et al. [2, 19]. Both works demonstrated how Deep Neural Networks could be applied to tasks that map sequential input data to sequential output data, giving the problem set its name: *sequence-to-sequence* (seq2seq).

Over the following years, seq2seq problems have been a major focus in research. Their importance also stems from the wide range of applications found for such problem sets. Seq2seq is relevant for natural language processing, spanning use cases such as machine translation, text summarization, and prominently dialog systems. It is also relevant for speech and audio problems in areas such as speech recognition, text-to-speech conversions, and other domains, such as code generation and time series forecasting [16, pp. 2f., 20–28].

The key challenge of seq2seq problems is capturing temporal dependencies. Sequential data typically has temporal and positional dependencies. The meaning of each element depends on its context and the arrangement relative to other elements of the sequence. Models must 'remember' relevant information from earlier in the sequence to make predictions about later elements [5, p. 367f.]. Furthermore, the distance between relevant elements can vary from short-range to long-range dependencies [24, p. 6f.]. The challenges of seq2seq problems can be described by two simple examples from natural language processing:

- **E1:** "The cat chased the mouse" vs.
  "The mouse chased the cat"
- **E2:** "I saw the man with the telescope"

E1 showcases the challenge of element arrangement. Both sentences use the same elements (words), but have a different meaning based on order alone. **E2** demonstrates contextual dependencies. The sentence has different meanings depending on the emphasis placed on different words. A model must capture these dependencies and interpret the sentence accordingly. Additionally, the context could be extended by previous or following sentences, introducing long-range dependencies across the sequence.

1.1 State-of-the-Art Paradigms

For seq2seq problems, two state-of-the-art architecture paradigms exist. The classical approach, introduced by Sutskever et al. and Cho et al., is the Recurrent Neural Network (RNN) [2, 19]. RNNs use a hidden state that accumulates information from previous elements of the input sequence. The hidden state is recurrently updated with each new element, allowing information to propagate sequentially through time [5, p. 372]. The modern solution is the Transformer model. Transformers employ an attention mechanism that relates every element in the sequence with every other element, enabling direct modeling of long-range dependencies. Unlike the hidden state of RNNs, Transformers capture global context through self-attention [24, pp. 1–4].

Both state-of-the-art solutions are designed for different use cases and offer different trade-offs, but achieve strong results for many applications. Furthermore, both paradigms have a numerous variations and improvements, often tailored for specific tasks or to address particular limitations. Core aspects of both paradigms are that RNNs process sequences efficiently during inference, but are expensive to train. Transformers excel at parallel training, but face quadratic complexity with sequence length [16, p. 8].

1.2 State Space Model Paradigm

A new State Space Model (SSM) paradigm was introduced between 2020-2023 through a series of works, resulting in models like S4 by Gu et al. and Mamba by Gu and Dao [6, 7]. Both SSMs are also designed specifically for seq2seq problems. They are based on State Space Representation adapted from control engineering, where systems are modeled using state variables that evolve over time. In control theory, SSMs are typically used to model dynamical systems and their state transitions over time based on previous states [12]. SSMs leverage first-order linear differential equations to model sequential data. The paradigm aims for efficient recurrent inference while enabling parallelizable training [7, p. 3f.]. This positions SSMs as a competitor to both RNNs and Transformers, representing a modern third solution that claims to combine advantages from both paradigms [6, 7].

1.3 Paper Goal

This paper provides an accessible introduction to State Space Models while critically evaluating their performance claims. Through conceptual explanation and empirical analysis, two questions are addressed:

(1) **How do SSMs work?** Provide step-by-step explanation of S4 and Mamba architectures, highlighting theoretical advantages: linear sequence complexity, parallel training via convolutional representation, and recurrent inference with constant memory.
(2) **Do SSMs outperform existing paradigms?** Examine published results across diverse benchmarks (long-range dependencies, language modeling, machine translation) comparing SSMs against RNNs and Transformers to identify task-specific strengths and limitations.

The analysis reveals SSMs as specialized competitors excelling in long-sequence and inference-critical tasks, complementing rather than replacing Transformers for general sequence modeling.

1.4 Paper Structure

The paper is structured as follows. Section 2 presents related work, providing context for SSM development. Section 3 covers the theoretical foundations, explaining RNNs, Transformers, and the evolution of SSM architectures in detail. Section 4 presents the comparative analysis, evaluating RNNs, Transformers, and SSMs across training and inference metrics. Finally, Section 5 concludes with key findings and reflections on the insights gained.

2 Related Work

Sutskever et al. and Cho et al. can be marked as a starting point in seq2seq learning, as stated in the introduction [2, 19]. However, the research on RNNs began earlier with the works by Rumelhart et al. and Elman [4, 17]. These early RNNs struggled with vanishing gradients, leading to overwhelming training complexity [9, pp. 1,3–6]. Long Short-Term Memory, introduced by Hochreiter and Schmidhuber, and Gated Recurrent Units by Cho et al. represented significant improvements over classical RNN architectures [3, 9]. Both LSTM and GRU are specialized types of RNN architectures designed to overcome the vanishing gradient problem and capture long-range dependencies in sequential data, where GRUs can be understood as a simpler mechanism compared to LSTMs. Cho et al. also introduced the encoder-decoder architectural pattern [3, p. 2f.]. Sutskever et al. applied the encoder-decoder LSTM architecture to machine translation. They showed that an RNN architecture could work for complex sequential tasks at scale. Their results demonstrated state-of-the-art performance that rivaled statistical machine translation systems [19, pp. 1ff.,6]. Together, these works established the encoder-decoder paradigm as a foundation for neural seq2seq learning.

The introduction of the Transformer architecture by Vaswand et al. marked a paradigm shift in seq2seq modeling [24]. Transformers rely on self-attention mechanisms to capture relationships between all pairs of elements, enabling parallel processing and direct modeling of long-range dependencies. This attention mechanism, however, has its trade-offs, notably in computational complexity [24, pp. 2–7]. The success of Transformer models motivated extensive research into efficient Transformer variants, such as Reformer, Sparse Transformer, BigBird, and Longformer. Each variant represents a trade-off between computational complexity and modeling capability. These variants, among others, demonstrate strong performance and represent active areas of research in addressing the efficiency challenges of Transformers [21]. However, a detailed analysis of these variants is beyond the scope of this paper.

This paper focusses on the alternative paradigm to Tansformers: SSMs. SSMs emerged from control theory and have their origin in the work on Linear State Space Layers by Gu et al [8]. Albert Gu can moreover be described as the founder of the SSM architecture and its main contributor. Gu et al. introduced the S4 model, which introduced structured parameterizations and HiPPO; an approach to capture long-range dependencies with linear complexity [7]. Mamba by Gu and Dao extended S4 with selective mechanisms that dynamically filter information [6].

3 Theoretical Foundations

3.1 Recurrent Neural Network

The core concept of an RNN is the use of recurrent processing loops to retain information while processing sequential data. An RNN processes each input element sequentially, where information from previous steps is preserved in a hidden state that is continuously updated. At each timestep, the computation depends only on the previous hidden state, introducing temporal dependency. Each step produces a new hidden state based on both the current input and the internal state. Consequently, a single hidden state accumulates past information over time. This mechanism is referred to as stateful computation [5, pp. 370,372–376], [13].

A simplified RNN cell is illustrated in Fig.1, highlighting the recursive structure and the unrolled processing over time [13].

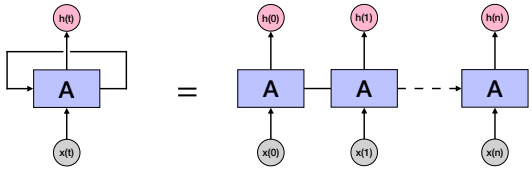

Figure 1: RNN structure recursive and unrolled. Adapted from [13].

The fundamental hidden state update is defined as shown in Eq. 1.

$$h^{(t)} = f\left(h^{(t-1)}, x^{(t)}; \theta\right) \tag{1}$$

3.1.1 Architecture. In the LSTM paradigm, each recurrent module consists of multiple interacting computation layers. The component is the cell state c_t. The previous state c_{t-1} is an input to a cell and c_t produced as an output at each element. A module has the ability to modify the cell state by adding or removing information through gates. A gate consists of a σ-activation followed by a point-wise multiplication operation. The layers function as follows: 1) *forget gate:* controls how much past information is retained, producing f_t. 2) *input gate:* determines how much information from the input x_t is written to memory, resulting in i_t. 3) *candidate gate:* creates candidate value vector C_t, resulting in what information to write. 4) *output gate:* updates hidden state h_t and final cell state c_t [13].

3.1.2 Training. RNNs are trained using Backpropagation Through Time, where gradients are propagated across sequential timesteps. Training an RNN has a time and memory complexity of $O(Ld^2)$ per sequence, where L is the sequence length and d the feature dimension. The memory complexity is $O(Ld)$, because hidden states must be stored for gradient computation. However, due to the sequential dependency of hidden states, computation cannot

be fully parallelized across timesteps, which results in a sequential bottleneck during training [13].

3.1.3 Inference. RNNs are also sequential during inference, with a time complexity of $O(Ld^2)$ per sequence of length L and d feature dimension. Memory requirements are minimal during inference, only requiring the previous hidden state h_{t-1} and cell state c_{t-1}, resulting in $O(d)$ [16, p. 7].

3.1.4 Common Extensions. Several common extensions that enhance the modeling capabilities of RNNs have been proposed. Bidirectional RNNs process sequences in both forward and backward directions, allowing the model to capture both past and future context. Encoder–decoder models support variable-length input and output sequences by encoding the input into a fixed-size context vector, which is then decoded autoregressively. Attention mechanisms provide direct access to relevant past hidden states, mitigating the limitations of standard RNNs in modeling long-range dependencies [5, pp. 388-392,412–415].

3.2 Transformer

The core concept of a Transformer model is the use of self-attention mechanisms. Transformers process all sequence elements in parallel. An entire input sequence is processed simultaneously, where each element influences all other elements to capture contextual relationships. This allows global interactions across the sequence, resulting in position-aware representations. As a result, the model maintains and enhances the global view of the sequence for each element, referred to as stateless computation [24, pp. 2–7]. A simplified Transformer encoder layer is illustrated in Fig. 2, highlighting the self-attention and feed-forward components.

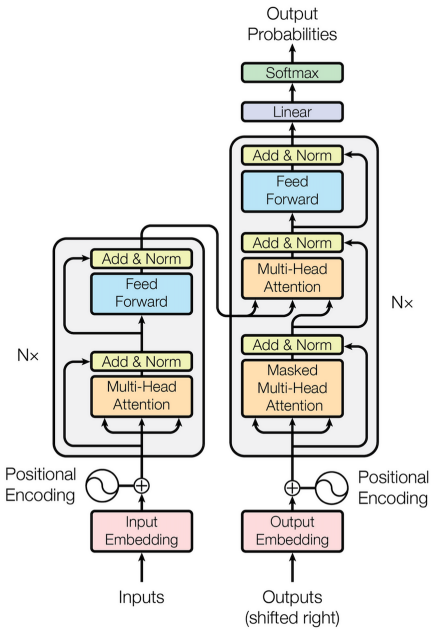

Figure 2: Transformer structure [24, p. 3].

The fundamental self-attention operation is defined as stated in Eq. equation 2. In the self-attention operation, each input element is projected into a query (Q), key (K), and value (V) vector.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{2}$$

3.2.1 Architecture. In the Transformer paradigm, each layer consists of two main components: multi-head self-attention and position-wise feed-forward networks. The core component is the attention mechanism. For each input position, the model computes queries (Q), keys (K), and values (V) through learned linear projections. Attention scores are computed by comparing each query with all keys, scaled by $\sqrt{d_k}$, and normalized via softmax [24, pp. 2–7]. These scores weight the values to produce the output representation. The layers function as follows: 1) *Multi-head attention:* projects inputs into multiple subspaces, computing attention in parallel across h heads. 2) *Add & Norm:* applies residual connections followed by layer normalization. 3) *Feed-forward:* processes each position independently through two linear transformations with activation. 4) *Add & Norm:* applies another residual connection and normalization [24, pp. 2–7].

3.2.2 Training. Transformers are trained using standard backpropagation, where gradients flow through all layers simultaneously. Training a Transformer has time complexity of $O(L^2d)$ per sequence, where L is sequence length and d is model dimension, due to the all-pairs attention computation. Memory complexity is $O(L^2 + Ld)$ for storing attention matrices. The parallel nature of self-attention enables full parallelization across sequence positions during training, which results in significant training efficiency compared to RNNs despite the quadratic complexity [24, p. 6].

3.2.3 Inference. Transformers process sequences in parallel in encoder-only models, with a per-layer time complexity of $O(L^2d)$ due to the quadratic cost of self-attention. In autoregressive decoding, generation proceeds sequentially. With key-value caching, the cost per generated token is $O(Ld)$, leading to a total inference complexity of $O(L^2d)$ for a sequence of length L.

Memory requirements during training are dominated by the attention matrix of size $O(L^2d)$. During autoregressive inference with caching, memory scales as $O(Ld)$ per layer due to stored key-value pairs [24, p. 6].

3.3 State Space Model

The operational concept of an SSM is similar to an RNN. SSMs also rely on sequential processing of input elements, where the contextual information required for processing each element is stored in a hidden state that is updated at each step. SSMs differ from RNNs in that the architecture is based solely on linear computations [7, pp. 1–4]. The core SSM architecture is built on four learnable parameters: Δ, A, B, C, which define the sequence-to-sequence transformation [7, p. 3]. A detailed and accessible exposition of the S4 model is provided in the annotated tutorial by Rush and Karamcheti [18].

3.3.1 State Space Representation. SSMs are rooted in continuous-time linear dynamical systems from control theory and signal processing. An SSM is based on three variables: $x(t) \in \mathbb{C}$ represents the n state variables, $u(t) \in \mathbb{C}$ represents the m state inputs, and $y(t) \in \mathbb{C}$ represents the p outputs. The SSM architecture is built on four learnable matrices: $\boldsymbol{A} \in \mathbb{C}^{nxn}$ is the state matrix, $\boldsymbol{B} \in \mathbb{C}^{nxm}$ is the control matrix, $\boldsymbol{C} \in \mathbb{C}^{pxn}$ is the output matrix, $\boldsymbol{D} \in \mathbb{C}^{pxm}$ is the command matrix [7, p. 3], [18].

The dynamics of an SSM are described by the following equation Eq. 3. $\dot{x}(t)$ represents the state, while $y(t)$ represents the respective output.

$$\begin{aligned} \dot{x}(t) &= Ax(t) + Bu(t), \\ y(t) &= Cx(t) + Du(t). \end{aligned} \tag{3}$$

These equations can be simplified. Since the time dependence is implicit, the (t) notation can be omitted. Additionally, for SSMs the feedthrough term $\boldsymbol{D}u(t) = 0$, which reduces the SSM equations further [18]. This leads to the equations shown in Eq. 4.

$$\begin{aligned} \dot{x}(t) &= Ax(t) + Bu(t), \\ y(t) &= Cx(t) \end{aligned} \tag{4}$$

Both equations describe the continuous-time model with only first-order differential equations. They capture how information flows through the system. The first equation (in Eq. 4) is responsible for the state update. It defines how the hidden state x evolves over time based on two factors: the current hidden state and a new input u. Matrix **A** determines how much past information is kept, while **B** controls how much new input is incorporated. The second equation (in Eq. 4) produces the model output. It extracts information of the hidden state by projecting it through matrix **C** [18].

3.3.2 Discretization. Neural networks do not operate with a continuous time function $u(t)$. The SSM must thus be adapted into a discrete-time SSM. This is achieved by introducing a step size Δ, that can be understood as a resolution of the input. With the step size, the discrete input is sampled to an implicit continuous signal $u(t)$, where $u_k = u(k\Delta)$. The discretization converts the state matrices into approximations [7, p. 4] [18]. Eq. 5 showcases this step.

$$\begin{aligned} \bar{A} &= \exp(\Delta A), \\ \bar{B} &= A^{-1}\big(\exp(\Delta A) - I\big)B, \\ \bar{C} &= C. \end{aligned} \tag{5}$$

Eq. 6 shows the discrete-time SSM equations that are used for sequence-to-sequence mapping instead of function-to-function.

$$\begin{aligned} x_k &= \bar{A}x_{k-1} + \bar{B}u_k, \\ y_k &= \bar{C}x_k \end{aligned} \tag{6}$$

3.3.3 HiPPO. Up to this point, the SSM framework can be applied to sequence-to-sequence learning problems, but naive implementations perform poorly in practice. Gu et al. identify that the core challenge lies in how the state matrix $\boldsymbol{A}$ is initialized: poorly chosen values lead to exponential decay or growth of gradients over long sequences, similar to the vanishing gradient problem in RNNs [7, p. 11].

The HiPPO (High-order Polynomial Projection Operators) framework addresses this by providing a specific initialization for $\boldsymbol{A}$. HiPPO theory defines a class of special matrices that enable the state $x(t)$ to optimally compress the history of inputs $u(t)$ using orthogonal polynomial bases. HiPPO aims to ensure that the hidden state maintains a detailed summary of past inputs instead of loosing past information arbitrarily [7, p. 4f.] [18]. The HiPPO matrix is shown in Eq. 7.

$$A_{n,k} = \begin{cases} -\sqrt{2n+1}\,\sqrt{2k+1}, & n > k, \\ -(n+1), & n = k, \\ 0, & n < k, \end{cases} \qquad n,k = 0,\dots,N-1 \tag{7}$$

3.3.4 S4. The previously introduced methods form the foundation of the Structured State Space Model by Gu et al. S4 combines discrete-time SSMs with the HiPPO matrix initialization, with an additional innovation: structured parameterization of the state matrix $\boldsymbol{A}$. S4 constraints $\boldsymbol{A}$ to have a specific, diagonal plus low-rank (DPLR) form. This structured parameterization leads to a reduced computational complexity in the sequential matrix multiplications [7, p. 5f.]. B is batch size, L the sequence length, D the model feature dimension, and N the state dimension. The S4 algorithm can be summarized as follows [6, p. 6]:

Algorithm 1 S4

Require: $x \in \mathbb{R}^{B\times L\times D}$
Ensure: $y \in \mathbb{R}^{B\times L\times D}$
1: $A \in \mathbb{R}^{D\times N} \leftarrow$ Parameter
▷ Represents structured $N \times N$ matrix
2: $B \in \mathbb{R}^{D\times N} \leftarrow$ Parameter
3: $C \in \mathbb{R}^{D\times N} \leftarrow$ Parameter
4: $\Delta \in \mathbb{R}^{D} \leftarrow$ Parameter
5: $(\bar{A}, \bar{B}) \leftarrow \text{discretize}(\Delta, A, B)$
6: $y \leftarrow \text{SSM}(\bar{A}, \bar{B}, C)(x)$
▷ Time-invariant: recurrence or convolution
7: **return** y

3.3.5 Convolutional Training. S4 introduces a dual representation of the SSM model. The same SSM can either be expressed as a recurrent sequential model or as a global convolution. The convolutional model is useful during training, as it eliminates the sequential bottleneck during training, that RNNs experience.

In the classical S4, the matrices $\boldsymbol{A}$, $\boldsymbol{B}$, and $\boldsymbol{C}$ are time-invariant. This means that they stay constant during the processing of all sequence elements and do not depend on the input content [7, p. 4f.]. This allows the SSM to be reformulated as a convolution with a fixed kernel $\boldsymbol{K}$, that can be precomputed The convolution kernel is computed as shown in Eq. 8.

$$\begin{aligned} x_0 &= \bar{B}u_0 & y_0 &= C\bar{B}u_0 \\ x_1 &= \bar{A}\bar{B}u_0 + \bar{B}u_1 & y_1 &= C\bar{A}\bar{B}u_0 + C\bar{B}u_1 \\ x_2 &= \bar{A}^2\bar{B}u_0 + \bar{A}\bar{B}u_1 + \bar{B}u_2 & y_2 &= C\bar{A}^2\bar{B}u_0 + C\bar{A}\bar{B}u_1 + C\bar{B}u_2 \\ &\vdots & &\vdots \end{aligned} \tag{8}$$

L is the sequence length. The output can be calculated using Eq. 9.

$$y = \mathbf{K} \cdot u \tag{9}$$

This convolution can be computed efficiently using FFTs in $O(L \log L \cdot d)$ time, enabling fully parallel training across the entire sequence [7, p. 23].

3.3.6 Recurrent Inference. During inference, the S4 model switches back to its recurrent mode. Output elements are generated sequentially, with each new token depending on the previous hidden state of the model. This recurrent formulation is memory-efficient, as only the current state vector needs to be stored, making memory usage independent of the sequence length L. Thanks to the diagonal structure of the state matrix in S4, the state update can be computed element-wise. As a result, the computational cost per step is constant, and generating a sequence of length L requires linear time in L [7, p. 7].

3.3.7 Mamba. Mamba is an enhancement of the S4 model. The structured parameterization with time-invariant matrices ($\boldsymbol{A}$, $\boldsymbol{B}$, $\boldsymbol{C}$) is a crucial factor in S4's dual representation mode, which allows the model to be computed convolutional and recurrently. Mamba relaxes this time-invariance constraint to achieve greater modeling capacity. The main contributions of Mamba are: 1) *Selective Scan:* allows the model to filter information differently 2) *Hardware-Awareness:* allows for efficient computation during training.

Mamba introduces selectivity by making the matrices $\boldsymbol{B}$, $\boldsymbol{C}$ and the step size Δ input dependent. Matrix $\boldsymbol{A}$ remains fixed, but is also dynamic through its modulation with Δ. The parameters also gain a additional dimension B, enabling different processing for each sequence in the batch. This selectivity allows Mamba to dynamically adjust its behavior. Different configurations allow it to retain long-range dependencies better or forget irrelevant context to focus on short-range dependencies.

However, this input-dependence breaks the convolutional representation of S4, requiring a new computational approach for efficient training. Naive recurrent implementations of Mamba would suffer from the sequential bottleneck that S4's convolution avoided. Mamba addresses this through a hardware-aware parallel scan algorithm specifically designed for GPU architectures. The scan algorithm optimizes the data transfer bottleneck in the high-bandwidth memory of the GPU, that were identified as a major factor in the runtime of the Mamba algorithm [6, pp. 2–9].

The high-level overview of the Mamba algorithm is as follows. In contrast to S4, $\boldsymbol{B}$, $\boldsymbol{C}$, and Δ are not constant anymore, but dependent on the input [6, p. 6].

Algorithm 2 Mamba (S6)

Require: $x \in \mathbb{R}^{B \times L \times D}$
Ensure: $y \in \mathbb{R}^{B \times L \times D}$
$A \in \mathbb{R}^{D \times N} \leftarrow$ Parameter
▷ Represents structured $N \times N$ matrix
$B \in \mathbb{R}^{B \times L \times N} \leftarrow s_B(x)$
$C \in \mathbb{R}^{B \times L \times N} \leftarrow s_C(x)$
$\Delta \in \mathbb{R}^{B \times L \times D} \leftarrow \tau_\Delta + s_\Delta(x)$
$(A, B) \leftarrow \text{discretize}(\Delta, A, B)$
$y \leftarrow \text{SSM}(A, B, C)(x)$
▷ Time-varying: recurrence (scan) only
return y

3.4 Theoretical Comparison

This section provides a theoretical comparison of RNNs, Transformers, and SSMs, focusing on their fundamental architectural differences rather than empirical performance. The comparison examines four key aspects: 1) *Memory*: how each architecture stores and maintains contextual information, 2) *Training Complexity*: computational costs and parallelization properties during training, 3) *Inference Complexity*: efficiency during autoregressive generation.

3.4.1 Memory. The architectures differ fundamentally in how they store contextual information. Transformers maintain the full context through a KV cache. Every key-value representation for all previous elements of the sequence is stored. This enables the model to have direct access to any position, preserving the complete information within the context. This comes at a cost of $O(L^2)$ attention computation and $O(Ld)$ memory for the KV cache, where L is the sequence length and d the hidden dimension.

In contrast, SSMs and RNNs utilize a fixed-size information storage. This state compresses the input sequence into a fixed dimension. The compressed representation consequently also requires $O(1)$ memory during inference (and $O(Ld)$ during training, because the model must store previous information for back propagation). However, while Transformers preserve the complete information, SSMs and RNNs are by design selective.

3.4.2 Training Complexity. Training complexity must be assessed under two aspects: per-token computational operations and the ability to parallelize across the sequence. In per-token operations, Transformers are the most expensive to train with $O(L^2 d)$, followed by S4 $O(L \log L \cdot d)$. The parallel scan algorithm of Mamba and RNN are similar with a linear complexity of $O(L)$ and $O(Ld)$.

However, the per-token complexity does not determine practical training efficiency. Transformers and S4 can process the entire sequence in parallel. Transformers achieve this through the independent attention computations, while the S4 paradigm allows the crucial convolutional representation. Mamba also achieves full parallelization despite its input-dependence through its hardware-aware parallel scan algorithm. RNNs, in contrast, are fundamentally sequential. Every hidden state depends on the previous hidden state, which prevents parallel computation across the sequence. This sequential bottleneck makes RNNs slower to train.

3.4.3 Inference Complexity. The inference complexity for autoregressive generation primarily depends on the computational cost per newly generated token. Transformers have a complexity of $O(Ld)$, where L is the length of the context and d is the dimension. For a full-sequence computation, this leads to a quadratic time-complexity of $O(L^2 d)$. This quadratic time-complexity becomes problematic with long sequences.

In contrast, RNNs, S4 and Mamba maintain constant complexity per step due to their compressed state representation. Each new token can be computed with a fixed amount of computation per step, making the total time complexity linear in L with $O(Ld)$.

3.4.4 Conclusion. In conclusion, the SSM paradigms S4 and Mamba effectively combine benefits from both RNNs and Transformers, while avoiding many of their drawbacks.

Architecturally, the SSM models follow the RNN paradigm through their compressed hidden state that accumulates sequence information over time. This represents the main fundamental difference to Transformers that store the full context. Thus SSMs theoretically outperform Transformers in their inference time-complexity, especially for long sequences. However, SSMs diverge from RNNs through their ability for dual computation modes. During training, both Mamba and S4 outperform RNNs, because

of their parallelization ability, which eliminates the sequential bottleneck.

The primary architectural trade-off for computational efficiency remains the information access pattern: compressed state or direct attention between all elements. Tb. 1 summarizes the theoretical discussion.

Table 1: Comparison of RNNs, Transformers, and SSMs for autoregressive sequence modeling. Blue entries indicate fast operations, while red entries indicate slow operations in comparison. Adapted from [16, p. 7].

Metric	RNN	Transformer	SSM (S4/Mamba)
Training	Sequential	Parallel	Parallel
Inference	Recurrent	Quadratic	Recurrent
Time	$O(Ld^2)$	$O(L^2d)$	$O(Ld)$
Memory	$O(d)$	$O(Ld)$	$O(d)$

4 Comparative Analysis

A comparative analysis predominantly on empirical performance data is inherently challenging. The previous theoretical comparison between the model paradigms allowed for objective conclusions on runtime and memory requirements. Measuring performance of a model depends on multiple interacting factors. Implementation details, model scale, optimization strategies, pretraining procedures, and the choice of benchmark all significantly influence reported results. Consequently, a performance comparison across architectures is often tied to specific experimental setups rather than reflecting architectural differences.

A unified study evaluating S4, Mamba, Transformers, and RNNs under identical conditions could not be identified. Furthermore, modern studies typically omit classical RNN baselines, reflecting their declining relevance in state-of-the-art applications. Where available, RNN results are considered for completeness, but they are no longer representative. A further gap exists in the direct comparison between S4 and its proposed successor, Mamba. Although Mamba is widely presented as an improvement over structured state space models such as S4, only limited studies evaluate both architectures under closely matched conditions. Instead, each model is typically assessed within distinct experimental frameworks, making a direct comparison difficult.

For this reason this study follows the historical and methodological development of structured state space models. S4 and Mamba are primarily interpreted within their respective evaluation contexts presented in literature. An overarching conclusion and connection between the models is attempted where possible.

4.1 Comparative Analysis of S4

The S4 paper of Gu et al. evalutes the proposed S4 model primarily on the Long Range Arena benchmark. In addition, it also presents more general problems, such as CIFAR-10 density estimation and WikiText-103 language modeling, to strengthen its argumentation [7, p. 30f.]. This work follows the proposed structure. First, the results of LRA are analyzed, and then the results are put in context with other works.

4.1.1 Long Range Arena. The LRA is a benchmark suite introduced by Tay et al. to evaluate Transformer models specifically on problems with long sequence length [20]. The self-set desired characteristics of LRA are that the benchmark should be general, simple, challenging, with specific focus on long inputs, probing diverse aspects, and accessible. The tasks include the following:

- **Long ListOps:** A sequence of mathematical expressions that are nested. Challenges the model to reason over long contexts and evaluate hierarchical structures.
- **Byte-Level Text Classification:** Classification of text that is stored as a sequence of bytes. The byte-level setup is deliberate, challenging the model to reason with compositional, unsegmented data.
- **Byte-Level Document Retrieval:** Challenges the model's ability to encode and store compressed representations that are useful for retrieval.
- **Image Classification:** Classification based on sequences of pixels. Challenges the model to learn the 2D spatial relations embedded in the data.
- **Pathfinder:** An image (32×32) with two points and a line consisting of dashes. Challenges the model to predict whether the line connects the points or not.
- **Pathfinder-X:** The Pathfinder problem extended to extreme size (128×128). Challenges the model to solve the same problem but with long sequence length.

From an evaluation perspective, LRA is an well suited, structured benchmark to evaluate S4. It allows for a direct comparison between S4 and Transformers over multiple testcases with a focus on long range dependencies, an area identified in the theoretical analysis as a potential strength of SSMs over Transformers.

The S4 paper by Gu et al. compares S4 against efficient Transformer models, of which a subset is used in this comparison [7, pp. 8,25f.]. The subset is selected based on the best performing Transformer models proposed in the paper to highlight the results of the S4 model. For RNN baselines, the results from Gu et al.'s later work, "Resurrecting Recurrent Neural Networks for Long Sequences" from 2023, are used [14, p. 21]. While this paper was published after S4 and references it, it provides multiple relevant RNN model approaches. A study of Yu and Erichson with a similar methodology is referenced for the Mamba results [26].

The S4 paper uses a controlled experimental setup. All Transformer models are configured with 4 layers, a hidden dimension of 256 with 4 heads and approximately 600k parameters. The S4 model is parameter-matched by keeping its depth and hidden dimension constant while adjusting only the state size (typcally $N = 64$). All models are trained using the AdamW optimizer with task-specific hyperparameter tuning [7, pp. 8,25f.]. The RNN models follow a similar training approach. They use 6 layers and are also tuned individually for every task. Like Transformers and S4, they also use AdamW and hyperparameter optimization. The authors emphasize that their experimental setup is comparable to the S4 paper, allowing for a comparison across architectures [14, p. 22f.]. Mamba is also follows a similar training configuration and evaluation protocol [26, p. 11f.].

While it cannot be verified that the configurations represent globally optimal solutions, the approach appears reasonable and consistent across model paradigms. The results of Mamba must be interpreted with additional caution, because they originate from a different source with different authors.

The reported results (Tb. 2) show a clear pattern: S4 outperforms Transformers and RNNs across all LRA tasks. S4 also outperforms Mamba. The performance gains vary in magnitude, ranging from moderate improvements on Retrieval and Pathfinder to substantial for Image and Text classification. Since LRA is specifically designed to capture long-range dependencies,

Table 2: LRA performance scores for Transformers, S4, and RNNs (linear and ReLU). Symbols: x = failed, $-$ = not tested. Adapted from [7, pp. 8,25f.], [14, p. 22f.], [26, p. 11f.].

Model	ListO	Text	Retr	Img	Path	Path-X
Transformer	36.37	64.27	57.46	42.44	71.40	x
Reformer	37.27	56.10	53.40	38.07	68.50	x
Performer	18.01	65.40	53.82	42.77	77.05	x
Lin-Transf.	16.13	65.90	53.09	42.34	75.30	x
S4	59.60	86.82	90.90	88.65	94.20	96.35
Mamba	38.02	82.98	72.14	69.82	69.26	67.32
RNN-Lin	50.40	89.10	89.10	–	x	x
RNN-ReLU	37.60	88.00	88.50	–	x	x

the findings support the effectiveness of S4 in this area. For the RNN baselines, Gu et al. find RRNs perform similar to S4. Their research also finds that the baseline RNN can be improved by dropping the nonlinearity of the RNN (RNN-Lin), which is surprising, because nonlinearity is a core architectural decision of the RNN paradigm.

Taken together, these results suggest that S4 matches Transformer performance on general sequence modeling tasks while demonstrating clear advantages on problems requiring long-range dependency modeling. This positions S4 as strong competitors to state-of-the-art sequence architectures. Crucially, Mamba, which can be seen as a successor to S4, does not outperform S4 on LRA. Its performance results are more similar to Transformers, suggesting that the proposed architectural improvements do not translate to LRA. This is a first indication that LRA results should be interpreted with caution, particularly when used as a primary performance measure of models for seq2seq problems.

4.1.2 LRA Training Protocol Critique. The paper "Never Train from Scratch: Fair Comparision of Long-Sequence Models Requires Data-Driven Priors", by Amos et al. argues whether the reported LRA performance of S4 actually reflects an architectural superiority or differences in the training methodology [1]. The paper argues that Transformers perform subpar in the paper of Gu et al., among other papers they explored. Amos et al. see the main problem in the training approach taken. They argue that pretrained models have shown strong performances in tasks involving long range dependencies [11, 22].

In their study, the models are first self pretrained using a denoising objective on the downstream task data, referred to "Masked SPT" by the authors, instead of being trained from random initialization. This is in contrast to the S4 paper, which avoids pretraining on a large dataset and instead trains all models from scratch. As a result, their experimental setups are not directly comparable. However, the authors closely follow the experimental structure of Gu et al. and explicitly position their work as a direct comparison, by conducting a test with similar model parameters and size. They also apply a similar pretraining approach to the S4 model of Gu et al [1, p. 5ff.].

The results of their study, presented in Tb. 3, clearly show that models pretrained with the denoising objective achieve substantially improved performance across all LRA tasks. A key finding is that the Transformer model exhibits particularly large gains and, after pretraining, performs on par with the S4 model. This outcome directly challenges the conclusions drawn from scratch-trained LRA evaluations discussed in the previous section. It suggests that the dominance of S4 is closely connected to the random-initialization training, while performance parity emerges when alternative training methodologies are employed.

Table 3: LRA performance score comparison between pretrained (Masked SPT) and scratch-trained models for Transformers and S4. Adapted from [1, p. 5].

Model	ListO	Text	Retr	Img	Path	Path-X
Transformer	36.37	64.27	57.46	42.44	71.40	x
Tr. Masked SPT	59.75	89.27	88.64	74.22	88.45	87.73
S4	59.60	86.82	90.90	88.65	94.20	96.35
S4 Masked SPT	61.25	90.34	88.74	89.36	94.92	96.94

4.1.3 S4 General Sequence Modeling Performance. Beyond LRA, the S4 paper evaluates general sequence modeling performance on additional benchmarks, including Speech Commands (SC10) classification, pixel-level 1D image classification, CIFAR-10 density estimation, and WikiText-103 language modeling [7, pp. 9f.,27–32].

For speech commands, Gu et al. report two evaluation results of S4. The original test uses a reduced 10-class subset of the 35-class SC speech command classification dataset. Their later analysis includes the full dataset of spoken English words, such as "up" or "yes", and the task requires to classify these clips correctly. While Gu. et al. recommend to use the full-dataset evaluation, the original test includes comparisons to RNN and Transformers. The results indicate that S4 outperforms both the strongest RNN baselines and Transformers, although, as noted in the previous critique, comparisons against Transformers should be interpreted cautiously due to differences in training methodology [7, pp. 9f.,27–32]. Their results are summarized in Tb. 4.

Table 4: SC10 classification (pre- processed MFCC features, unprocessed raw signals, frequency change). Symbol: x = failed. Adapted from [7, pp. 9f.,27–32].

Model	MFCC	Raw	0.5×
Transformer	90.75	x	x
Performer	80.85	30.77	30.68
ODE-RNN	65.90	x	x
ExpRNN	82.13	11.60	10.80
S4	93.96	98.32	96.30

For pixel-level 1D image classification and CIFAR-10 density estimation, S4 performs comparably to Transformers, with only minimal differences observed for image classification. These tasks involve learning local and global patterns over sequences of structured data, but do not require fine-grained content-based reasoning. In WikiText-103 language modeling, S4 again achieves performance on par with Transformers and surpasses all RNN architectures included in the benchmark. Collectively, these results indicate that S4 excels on a range of sequence modeling tasks that emphasize long-range dependencies or structured sequential input, such as audio classification and language modeling, while tasks requiring precise alignment or dense content reasoning remain more challenging.

4.1.4 S4 critique in Machine Translation. A major critique of the S4 model architecture comes from Vardasbi et al. in "State Spaces Aren't Enough: Machine Translation Needs Attention" from 2023 [23]. The study evaluates S4-based architectures on machine translation tasks, specifically English→German and English→Romanian. It also includes hybrid S4–Transformer models and a special S4 varaint S4*a* that is argumented with an attention mechanism. The models are trained with extensive hyperparameter optimization to maximize performance.

Tb. 5 summarizes the English–German results for standard S4, Transformer, and S4*a*. The performance is measured using BLEU scores, with results reported both by sentence-length buckets and overall. BLEU compares the model output to a referenced translation of the work.

Table 5: Machine translation performance on WMT'14 EN-DE set evaluated by BLEU score. Adapted from [23, p. 8].

Model	Short [1,17]	Medium [18,29]	Long [30,117]	Overall
∅-S4	24.0	24.3	21.4	22.7
Tr-Tr	25.9	26.8	26.4	26.4
S4-S4*a*	25.0	26.5	25.3	25.6
Tr-S4*a*	25.6	26.9	26.5	26.5

The findings show that a pure S4 model underperforms Transformers across all sentence lengths, with the performance gap widening for longer sentences. In contrast, S4*a* does not show the same drop in performance. This indicates that the limitation stems from S4's fixed-size internal representation, which must compress the entire prior context, including both the source sentence and previous output tokens, which the authors explicitly mention. These results suggest that while S4 captures long-range dependencies effectively, it struggles with tasks that require fine-grained, content-sensitive reasoning.

Other literature and studies base adapt this view, but crucially do not provide additional information or showcase other benchmarks. A consensus in the literature is that while S4 excels on tasks involving very long sequences, such as audio modeling or synthetic long-context benchmarks, it struggles in settings that require dense information processing and explicit content-based reasoning. In particular, tasks such as machine translation highlight these limitations, as they require flexible, token-level interactions between source and target representations that are more naturally handled by attention mechanisms due to its broader and creater context through attention [10, 23, 25].

4.2 Comparative Analysis of Mamba

The Mamba paper of Gu and Dao evaluates the Mamba model more broadly compared to S4 in its respective paper. The evaluation begins with synthetic tasks, followed by large-scale language modeling and DNA modeling, and concludes with audio signal processing. These domains are not arbitrary: they represent application areas where Transformer-based models generally perform well. The study thus positions Mamba as a direct competitor to Transformer models. A critique of the study is, that Mamba is not evaluated against a consistent set of other models. Not all benchmarks include Transformers or similar Transformer models.

This analysis largely maintains a similar structure. It also begins with synthetic tasks, before discussing language modeling and machine translation. While S4 performed competitively in language modeling, it struggled with machine translation due to its underlying architecture. Mamba aims to improve on the S4 architecture. An evaluation thus allows to assess whether an improvement is visible in the benchmarks. Finally, the analysis focuses on long-sequence modeling tasks, as such tasks are frequently cited as a strength of SSMs.

4.2.1 Synthetic Tasks. Gu and Dao conduct two synthetic benchmarks in their work: induction heads and selective copying.

Induction head by Olsson et al. evaluates the model's ability to perform associative recall and in-context learning. The model should detect pattern in sequences that it later uses to predict output elements. Their study setting evaluates Mamba against several Transformer variants (MHA-Absolute, MHA-RoPE, MHA-xPos) as well as improved S4-derived architectures (H3, Hyena), but not S4 directly. The reported results show that Mamba performs competitively across sequence lengths and remains stable even for very long test sequences. All reference models experience a significant performance drop with an increasing sequence length up to a complete loss of accuracy. Fig. 3 showcases the results.

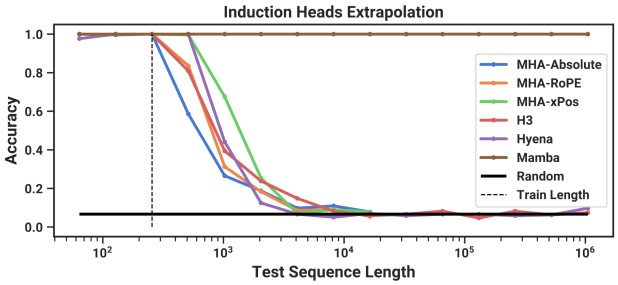

Figure 3: Induction heads benchmark [6, p. 11].

The second synthetic benchmark, selective copying, tests a similar ability. The model must retrieve and reproduce specific elements from earlier in the sentence. This benchmark by Gu and Dao is compared only against S4 and related S4 architectures (H3 and Hyena), but not Transformers. Mamba significantly outperforms these models, in some settings by more than 70%, which the authors attribute primarily to the input-dependent gating mechanism in the Mamba layer. However, the absence of Transformer references is noteworthy. The study "Repeat After Me: Transformers are Better than State Space Models at Copying" by Jelassi et al. directly addresses selective copying between Mamba and Transformer models [10]. In their copying benchmarks, Mamba also performs strongly but is generally outperformed by Transformer architectures using various positional encoding schemes (RoPE, NoPE, ALiBi, H-ALiBi). Moreover, Mamba's accuracy tends decreases with increasing sequence length in these experiments, whereas Transformers maintain more stable performance [10, pp. 6–9].

Other works conclude with similar ambiguous results, for example the studies listed for S4 in the previous sections. There is a consensus that Mamba generally outperforms S4 architectures. For some benchmarks, such as induction heads, Mamba appears get better benchmark results. Other tests, for example selective copying, show contradicting results, where Transformers outperform Mamba for specific tasks.

4.2.2 Language Modeling and Machine Translation. Gu and Dao also conduct extensive language modeling tests that compare

Mamba to a wide variety of S4 and Transformer architectures. For this summary, Hybrid H3, an improvement architecture based on S4, and Pythia, a pretrained Transformer model, are particularly relevant. The test does not include base Transformer or S4 models, however. the benchmark includes pretraining metrics (Perplexity) and Zero-Shot Evaluation, additionally, different model sizes from 140M to 2.8B parameters (for Mamba) in 5 size classes care compared on a variety of datasets: LAMBADA, HellaSwag, PIQA, ARC-challenge, ARC-easy: an easy subset of ARC-challenge, WinoGrande. The details are beyond the scope of this analysis, but the general conclusions are that that Mamba matches or outperforms the Transformer references for the respective parameter classes and is often comparable to Transformers of larger sizes [6, pp. 11f.,31f.]. Tb. 6 shows (parts of) the results in reduced form for Zero-Shot Evaluations. Average is kept from Gu et al. and not adapted.

Table 6: Zero-Shot Evaluations performance results for Hybrid H3, Pythia, and Mamba. Adapted from [6, pp. 11f.,31f.].

Model	Size	LAMBADA	PIQA	WinoGrande	Avg
Hybrid H3	130M	25.77	64.2	50.6	40.1
Pythia	160M	33.0	61.4	51.9	40.6
Mamba	130M	44.3	64.5	51.9	44.7
Hybrid H3	360M	48.0	68.1	54.1	48.0
Pythia	410M	51.4	66.9	53.8	48.2
Mamba	370M	55.6	69.5	55.3	50.0
Hybrid H3	1.3B	49.6	71.3	56.9	53.0
Pythia	1.4B	61.7	71.0	57.2	55.2
Mamba	1.4B	64.9	74.2	61.5	59.7
Hybrid H3	2.7B	55.7	71.3	61.4	58.0
Pythia	2.8B	64.7	74.0	59.7	59.1
Mamba	2.8B	69.2	75.2	63.5	63.3

In this context, machine translation is an interesting benchmark to compare. Both tests are in a similar domain, but with different focuses and requiring different abilities. A comparable machine translation benchmark is by Pitorro et al., however, a direct comparison can again only be conducted with extreme caution [15]. Their work is also too extensive to cover in detail. This analysis summarizes their key points for this evaluation. Pitorro et al. conduct a machine translation in between German and English with different datasets for Mamba and a variety of Transformer models. The relevant models in this context are only comparable in parts to Gu and Dao: Transformer architectures (encoder-decoder; based on best known architecture by Pitorro et al.), Mamba, Mamba-MHA (Mamba with added Multihead-Attention), Pythia. Mamba and Pythia are tested in two parameter configurations comparable to Gu and Dao: 430M (small) and 1.4B (medium) [15, p. 3ff.].

The results in Tb. 7 show that for the trained-from-scratch models, Mamba is competitive to both Transformers. The Mamba architecture with the added multihead-attention layer improves the accuracy further for both metrics. The results for pretrained models is in line with the training protocol critique by Amos et al. for the S4 benchmark results [1]. The Transformer model Pythia improves the accuracy significantly. The performance of Mamba also increases with pretraining, but less overall. Pitorro et al. explicitly state in their conclusion that pretraining plays a major role in the accuracy of the model. They also note that the performance differences shrink with scale and that data quality is also an important, maybe overlooked factor [15, p. 6ff.]. An analysis of the model performance to sequence length from their published results also reveals interesting details. Their results are that all models generally experience performance drops with increasing sequence lengths. Only special Encoder-Decoder architecture patterns, both for Mamba and Transformers, do not experience this drop. The results also show that pretraining helps Mamba and Transformers in their ability to process long sequences [15, p. 7].

Table 7: WMT23-CAT-10 BLEU scores for DE→EN and EN→DE. Adapted from [15, p. 17].

Model	DE→EN	EN→DE
Trained from Scratch		
Transformer Enc-Dec	28.3	29.3
Transformer (best)	29.8	29.1
Mamba	25.9	25.5
Mamba-MHA	27.8	25.9
Mamba Enc-Dec	31.4	30.1
Finetuned		
Mamba (430M)	25.6	22.5
Pythia-S (430M)	26.8	29.3
Mamba-M (1.4B)	32.5	27.5
Pythia-M (1.4B)	33.4	33.9

Overall, Mamba demonstrates particularly strong performance in language modeling, where it matches or slightly exceeds comparable Transformer models and generally improves S4. This supports the view that input-dependent state updates enhance contextual modeling relative to fixed-parameter SSMs. However, machine translation provides a more demanding evaluation setting. Translation requires fine-grained, token-level interactions between source and target sequences and flexible alignment across long contexts. In this setting, Transformer architectures remain highly competitive, likely due to their direct content-based attention mechanism. The Mamba-MHA model suggest that attention mechanisms retain advantages in tasks requiring dense, explicit cross-token interactions.

4.2.3 Very-Long Sequence Modeling. Gu and Dao further evaluate Mamba on multiple specific tasks that are designed for extremely long sequences. Such benchmarks are of interest, because Mamba should excel in such specific scenarios based of its single compressed hidden state architecture. The tests are: DNA sequence modeling and audio waveform processing. Unlike the previous benchmarks, these tests cannot be evaluated in combination with other studies or references. The tests are further specific and detailed. This section only wants to give a general high-level overview of the proposed results.

For DNA sequence modeling, Mamba is trained on genomic sequences of the Human Reference Genome, with lengths of over 1 million elements. Their results are that Mamba achieves competitive performance with specialized Transformer variants designed for long contexts, while maintaining significantly lower memory requirements during both training and inference [6, p. 12f.].

The audio modeling task has similar characteristics. The speech command dataset SC09 provides over 16.000 samples of one-second audio clips with digits from zero to nine with highly variable characteristics. Mamba must classify these clips correctly. Gu and Dao benchmark Mamba again against other state-of-the-art GAN-based and diffusion-based models. The results are that Mamba outperforms these models consistently, even with comparatively smaller models. Mamba also outperforms the baseline models on throughput [6, p. 14f.,34ff.].

These results suggest with a consistent pattern that Mamba performs well on sequences with extreme length, a domain where Transformers historically struggled due to their inherent model architecture. These results thus provide promising baseline for a number of new potential problem domains for Mamba.

4.3 Overall Comparative Summary

The results of this analysis reveal interesting details in multiple ways, but also critical aspects to consider - not only for the models and their performances, but also for an evaluation and comparison. This section wants to summarize these aspects.

4.3.1 The Role of Benchmark Methodology. A critical finding of this analysis is the difficulty of fairly assessing and comparing different studies and their results. The presented benchmarks and results are selected with best intentions and compared objectively. However, all studies rely on individual model configurations, training procedures, and evaluation parameters, making it difficult to evaluate model performance beyond individual experimental contexts. This has broader implications: claimed architectural benefits revealed in studies may be influenced by experimental methodology, as demonstrated by the significant role of pretraining in LRA benchmark results.

4.3.2 Task-Specific Performance Patterns. The comparative analysis reveals that RNNs, S4, Mamba, and Transformers excel at different benchmarks, with no model universally outperforming the others. Notably, the strengths and weaknesses of SSMs are highly task-specific and do not allow for overarching conclusions within a problem domain. For Mamba, the synthetic tests of induction heads and selective copying demonstrate this: both are in a similar domain, yet comparative results show nearly opposite performance patterns between Mamba and Transformers. For S4, detailed examination of LRA results similarly shows that no model outperforms others across all tasks. It is easy to draw overarching conclusions based on incomplete information from specific benchmark subsets. Consequently, other studies not included in this paper could shift the discussion in different directions.

4.3.3 Architectural Trade-offs Revisited. The empirical analysis confirms the theoretical trade-offs identified in Section 3.4, where the fundamental distinction between compressed state and full attention manifests consistently across tasks. The compressed state of SSMs enables constant memory during inference and linear complexity in sequence length, generally providing advantages for very long sequences (DNA, audio modeling) and deployment scenarios with memory or speed constraints. However, this compression necessarily loses information. For tasks that require fine-grained and content-specific reasoning, the compressed state shows limits, as shown in the performance of S4 for machine translations. Both architectures achieve parallel training through different mechanisms (Transformers via attention, S4 via convolution, Mamba via parallel scan), confirming that modern SSMs overcome the RNN sequential bottleneck. The task-specific performance patterns validate interpreting SSMs as specialized rather than general-purpose replacements: they excel for long sequences and inference-critical deployment, while Transformers remain superior for tasks that require in-depth attention across the input sequence.

5 Conclusion

5.1 Research Questions

5.1.1 How do SSMs work? SSMs originate from continuous State Space Representations that are discretized for neural network applications. The core architecture consists of learnable state matrices ($\boldsymbol{A}$, $\boldsymbol{B}$, $\boldsymbol{C}$) and a discretization parameter (Δ) that process sequences by accumulating information in a single hidden state. S4 improves upon naive SSMs through HiPPO initialization, enabling stable learning of long-range dependencies. Mamba extends S4 by making these parameters input-dependent for greater modeling flexibility, addressing the resulting training inefficiency through hardware-aware parallel scan algorithms. SSMs share the recurrent structure of RNNs but differ fundamentally in their linearity, which enables dual computational modes: a convolutional representation for parallel training and a recurrent mode for efficient inference. Unlike Transformers, which maintain full attention between all sequence elements, SSMs compress information into a single hidden state. This architectural difference trades context breadth for computational efficiency, enabling constant-time inference on extremely long sequences—a historically difficult domain for Transformers with their quadratic attention complexity.

5.1.2 Do SSMs outperform existing paradigms? The answer is nuanced. The SSM paradigm demonstrates theoretical advantages over both RNN and Transformer: parallelizable training (versus RNNs) and efficient inference via compressed state (versus Transformers). The comparative analysis reveals that S4 and Mamba perform competitively with state-of-the-art models across various domains, with Mamba generally outperforming S4. Mamba functions as a direct competitor to Transformers with no obvious drastic drawbacks, though improvements are incremental and task-specific, particularly for long sequences. It struggles in comparison with fine-grained, token-level interactions that require detailed previous sequential information. However, benchmark results across all models prove highly task-dependent and difficult to generalize, with performance determined more by task characteristics than by inherent architectural superiority. Mamba should be considered for new applications, but Transformer still is the proven, state-of-the-art paradigm for sequence-to-sequence problems.

5.2 Reflection and Limitations

The research questions' conclusions reflect the analysis overall: SSMs represent a novel paradigm that rivals rather than replaces Transformers. The evaluated benchmarks yield both promising and frustrating results, as no clear overall conclusion emerges. This analysis attempts objectivity and comprehensiveness but has clear limitations. Not all relevant studies could be incorporated, and as stated in Section 4.3.2, results are heavily influenced by study methodology and are highly task-specific. This work cannot examine all benchmark tasks in sufficient detail, and additional benchmarks could shift conclusions in different directions.

Furthermore, direct model comparisons are often impossible because models are not evaluated uniformly even within the preselected benchmarks. RNNs are frequently excluded from recent studies, even with ongoing research effort. A detailed comparison between modern RNN solutions and SSMs would be especially interesting, because of their similar paradigm approach. This paper acknowledges these gaps and the fact that they complicate an objective analysis. As a critical analysis, this work tries to put S4 and Mamba's claims into context and compares them against other studies to provide a balanced view across benchmark domains.

References

[1] Ido Amos, Jonathan Berant, and Ankit Gupta. 2024. Never Train from Scratch: Fair Comparison of Long-Sequence Models Requires Data-Driven Priors. arXiv:2310.02980 [cs.LG] https://arxiv.org/abs/2310.02980

[2] Kyunghyun Cho, Bart van Merrienboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. arXiv:1406.1078 [cs.CL] https://arxiv.org/abs/1406.1078

[3] Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014. Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078* (2014).

[4] Jeffrey L Elman. 1990. Finding structure in time. *Cognitive science* 14, 2 (1990), 179–211.

[5] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. 2016. *Deep Learning*. MIT Press. http://www.deeplearningbook.org.

[6] Albert Gu and Tri Dao. 2024. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv:2312.00752 [cs.LG] https://arxiv.org/abs/2312.00752

[7] Albert Gu, Karan Goel, and Christopher Ré. 2022. Efficiently Modeling Long Sequences with Structured State Spaces. arXiv:2111.00396 [cs.LG] https://arxiv.org/abs/2111.00396

[8] Albert Gu, Isys Johnson, Karan Goel, Khaled Saab, Tri Dao, Atri Rudra, and Christopher Ré. 2021. Combining Recurrent, Convolutional, and Continuous-time Models with Linear State-Space Layers. arXiv:2110.13985 [cs.LG] https://arxiv.org/abs/2110.13985

[9] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural computation* 9, 8 (1997), 1735–1780.

[10] Samy Jelassi, David Brandfonbrener, Sham M. Kakade, and Eran Malach. 2024. Repeat After Me: Transformers are Better than State Space Models at Copying. arXiv:2402.01032 [cs.LG] https://arxiv.org/abs/2402.01032

[11] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, et al. 2021. Highly accurate protein structure prediction with AlphaFold. *nature* 596, 7873 (2021), 583–589.

[12] Rudolph Emil Kalman. 1960. A new approach to linear filtering and prediction problems. (1960).

[13] Christopher Olah. 2015. Understanding LSTM Networks. https://colah.github.io/posts/2015-08-Understanding-LSTMs/. Accessed: 2026-02-01.

[14] Antonio Orvieto, Samuel L Smith, Albert Gu, Anushan Fernando, Caglar Gulcehre, Razvan Pascanu, and Soham De. 2023. Resurrecting Recurrent Neural Networks for Long Sequences. arXiv:2303.06349 [cs.LG] https://arxiv.org/abs/2303.06349

[15] Hugo Pitorro, Pavlo Vasylenko, Marcos Treviso, and André F. T. Martins. 2024. How Effective are State Space Models for Machine Translation? arXiv:2407.05489 [cs.CL] https://arxiv.org/abs/2407.05489

[16] Haohao Qu, Liangbo Ning, Rui An, Wenqi Fan, Tyler Derr, Hui Liu, Xin Xu, and Qing Li. 2025. A Survey of Mamba. arXiv:2408.01129 [cs.LG] https://arxiv.org/abs/2408.01129

[17] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. 1986. Learning representations by back-propagating errors. *nature* 323, 6088 (1986), 533–536.

[18] Sasha Rush and Sidd Karamcheti. 2022. The Annotated S4: Efficiently Modeling Long Sequences with Structured State Spaces. https://srush.github.io/annotated-s4/. Accessed: 2025-02-03.

[19] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. 2014. Sequence to Sequence Learning with Neural Networks. arXiv:1409.3215 [cs.CL] https://arxiv.org/abs/1409.3215

[20] Yi Tay, Mostafa Dehghani, Samira Abnar, Yikang Shen, Dara Bahri, Philip Pham, Jinfeng Rao, Liu Yang, Sebastian Ruder, and Donald Metzler. 2020. Long Range Arena: A Benchmark for Efficient Transformers. arXiv:2011.04006 [cs.LG] https://arxiv.org/abs/2011.04006

[21] Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. 2022. Efficient Transformers: A Survey. arXiv:2009.06732 [cs.LG] https://arxiv.org/abs/2009.06732

[22] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971 [cs.CL] https://arxiv.org/abs/2302.13971

[23] Ali Vardasbi, Telmo Pessoa Pires, Robin M. Schmidt, and Stephan Peitz. 2023. State Spaces Aren't Enough: Machine Translation Needs Attention. arXiv:2304.12776 [cs.CL] https://arxiv.org/abs/2304.12776

[24] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2023. Attention Is All You Need. arXiv:1706.03762 [cs.CL] https://arxiv.org/abs/1706.03762

[25] Jesse Vig and Yonatan Belinkov. 2019. Analyzing the Structure of Attention in a Transformer Language Model. arXiv:1906.04284 [cs.CL] https://arxiv.org/abs/1906.04284

[26] Annan Yu and N. Benjamin Erichson. 2025. Block-Biased Mamba for Long-Range Sequence Processing. arXiv:2505.09022 [cs.LG] https://arxiv.org/abs/2505.09022